

RAFAELA SOUSA DE ALENCAR LACERDA

**Estratégias de controle distribuído para sistemas de armazenamento em
baterias para regulação de tensão em redes de distribuição com alta
penetração de geração fotovoltaica**

R1.3: Configuração do ambiente

Orientador: Prof. Dr. Juan Carlos Cebrian

São Paulo
2026

1. INTRODUÇÃO

O OpenDSS (*Open Distribution System Simulator*) é um simulador de propósito geral para análise de sistemas de distribuição de energia elétrica, também amplamente utilizado para a simulação de fluxo de potência. Esse programa foi originalmente concebido para realizar praticamente todas as análises elétricas necessárias para o planejamento de redes de distribuição com Geração Distribuída (GD). Para tanto, a matriz de admitância do sistema, que relaciona correntes injetadas e tensões nodais, é formada pelas matrizes de admitância nodais primitivas de cada elemento do sistema, e suas equações são resolvidas por meio de um solver de matrizes esparsas. O fluxo de potência, cálculo que determina as tensões em cada barra e os fluxos de potência ativa e reativa nas linhas, é calculado de forma iterativa, utilizando o método Normal de injeção de corrente como algoritmo padrão, com o método de Newton disponível para circuitos de convergência mais difícil. Determinar o fluxo de potência é fundamental para avaliar o carregamento da rede, identificar violações de tensão e apoiar decisões de controle e planejamento.

A integração do OpenDSS com linguagens de programação de alto nível, como Python, possibilita o desenvolvimento de ferramentas computacionais sofisticadas que exploram os resultados do fluxo de potência de forma programática e automatizada. Dentre as aplicações possíveis, no contexto deste projeto, estudos relacionados à aplicação e modelamento de controladores de BESS para regulação de tensão podem ser realizados em diferentes cenários de operação.

Para viabilizar essa integração com Python, estão disponíveis diferentes interfaces de programação de aplicação (APIs). A abordagem oficial baseia-se na API COM (*Component Object Model*), de uso restrito ao Windows. Como alternativa multiplataforma, o projeto DSS-Extensions disponibiliza três bibliotecas Python: DSS-Python, OpenDSSDirect.py e AltDSS-Python, todas construídas sobre um motor compatível com o OpenDSS oficial. A escolha da API e suas particularidades serão detalhadas no desenvolvimento deste relatório.

Nesse contexto, esta etapa contempla a configuração do ambiente de simulação, por meio do desenvolvimento de um script em Python. Tal script executa o fluxo de potência da rede de distribuição padrão IEEE 33 barras, definida em arquivo com extensão .dss, e exporta os resultados relativos às tensões de fase em cada barra, às

correntes nas linhas e aos fluxos de potência ativa e reativa. Assim, constitui a interface básica para as etapas de controle a serem desenvolvidas.

2. API PYTHON

Para melhor definir o ambiente de simulação, as APIs Python do OpenDSS disponíveis foram analisadas. A API COM é acessível pelos pacotes Python *comtypes* ou *win32com*, porém é compatível somente com Windows, o que limita sua aplicabilidade em diferentes sistemas operacionais. As três bibliotecas Python do DSS-Extensions são construídas sobre uma reimplementação do motor oficial do OpenDSS em biblioteca própria, o *AltDSS engine*, e têm suporte multiplataforma (Windows, Linux e macOS):

- DSS-Python (pacote *dss*): reproduz a organização da API COM oficial, permitindo a migração de código com alterações mínimas. O acesso às propriedades e métodos segue a mesma hierarquia encadeada da interface COM (por exemplo, *dss.ActiveCircuit.ActiveBus.puVmagAngle*), de modo que usuários familiarizados com a interface oficial podem adotar essa biblioteca sem reescrever todo o código, apenas adaptando-o. Por utilizar o *AltDSS engine*, oferece suporte multiplataforma e melhor desempenho em comparação à implementação COM original.
- OpenDSSDirect.py (pacote *opendssdirect*): também construída sobre o *AltDSS engine*, expõe suas funcionalidades por meio de chamadas de função com organização hierárquica mais plana em relação à API COM, ou seja, com menos níveis de objetos encadeados para acessar as mesmas informações, como em *dss.Bus.puVmagAngle()*. Possui funções utilitárias adicionais (*opendssdirect.utils*) e suporta o uso opcional da biblioteca *pandas* como dependência recomendada, para facilitar a manipulação dos resultados de simulação.
- AltDSS-Python (pacote *altdss*): oferece acesso direto a quase todos os objetos do OpenDSS como objetos Python persistentes, com acesso às suas propriedades pelos mesmos nomes utilizados nos scripts .dss, além de funcionalidades que não existem nas outras APIs, como operações em lote sobre múltiplos elementos simultaneamente. Dessa forma, representa uma abordagem mais moderna e abrangente, facilitando a

migração entre interface gráfica (OpenDSS) e API, mas ainda se encontra em estágio inicial de desenvolvimento.

Diante dessas alternativas, a biblioteca `OpenDSSDirect.py` foi escolhida para compor o ambiente de simulação pelos seguintes motivos. Em primeiro lugar, trata-se do pacote mais maduro entre as opções do DSS-Extensions, encontrando-se na versão 0.9.4, enquanto o *altdss* está na versão 0.2.4, com API ainda sujeita a alterações. Em segundo lugar, sua interface por chamadas de função é direta e bem documentada, tornando o código mais legível em aplicações que realizam iterações frequentes sobre elementos da rede, como as simulações de controle previstas nas etapas subsequentes deste trabalho. Por fim, o módulo *opendssdirect.utils* disponibiliza funções utilitárias para conversão direta dos elementos da rede em DataFrames do *pandas*, facilitando a exportação e análise dos resultados do fluxo de potência. Dessa forma, o `OpenDSSDirect.py` apresenta o melhor equilíbrio entre maturidade, desempenho e facilidade de uso para os objetivos deste projeto.

3. SCRIPT EM PYTHON

O script desenvolvido, disponível em [1], tem como objetivo, primeiro, executar o fluxo de potência da rede padrão IEEE 33 barras, cujo modelo em formato .dss está disponível em [2] e, segundo, exportar os resultados em arquivos .csv para uso nas etapas subsequentes do projeto. Esse script é estruturado em três etapas principais.

Etapa 1: o ambiente de simulação é inicializado por meio da limpeza do estado interno do simulador (*dss.Basic.ClearAll()*) e do carregamento do arquivo de definição da rede (*dss.Command("Redirect \"IEEE33.dss\")*), que contém a topologia, os parâmetros das linhas e as cargas do sistema IEEE 33 barras.

Etapa 2: são coletadas as tensões de fase em cada barra. O script itera sobre todas as barras do circuito, obtendo, para cada nó, a tensão em valores por unidade (p.u.), o ângulo de fase em graus e a tensão base em kV, por meio das funções *dss.Bus.puVmagAngle()* e *dss.Bus.kVBase()*. Os resultados são armazenados em um DataFrame e exportados para o arquivo *tensoes_barras.csv*.

Etapa 3: são coletadas as correntes e potências nos elementos da rede. O script itera sobre as fontes de tensão (*Vsource*) e sobre todas as linhas (*Line*), extraíndo, para cada elemento, a magnitude e o ângulo da corrente na extremidade de envio, bem como as potências ativa (kW), reativa (kVAR) e aparente (kVA), por meio das

funções `dss.CktElement.CurrentsMagAng()` e `dss.CktElement.Powers()`. Esses resultados são exportados para o arquivo `correntes_linhas.csv`.

Os resultados obtidos constituem a base de dados necessária para as etapas de controle a serem desenvolvidas nas fases subsequentes do projeto, nas quais os valores de tensão nominais nas barras após a execução do fluxo de potência serão utilizados como parâmetros de referência para os algoritmos de controle de BESS.

3.1. Rede IEEE 33 barras

A rede IEEE 33 barras é um sistema de distribuição radial de referência amplamente utilizado na literatura técnica, apesar de não ser uma rede de testes oficial do IEEE, para estudos de fluxo de potência, reconfiguração de redes, alocação de geradores distribuídos e estratégias de controle de sistemas de armazenamento. Proposta originalmente por Baran & Wu [3], a rede possui tensão nominal de 12,66 kV, configurando-se como rede de média tensão, e é composta por 33 barras e 32 ramais, conforme a

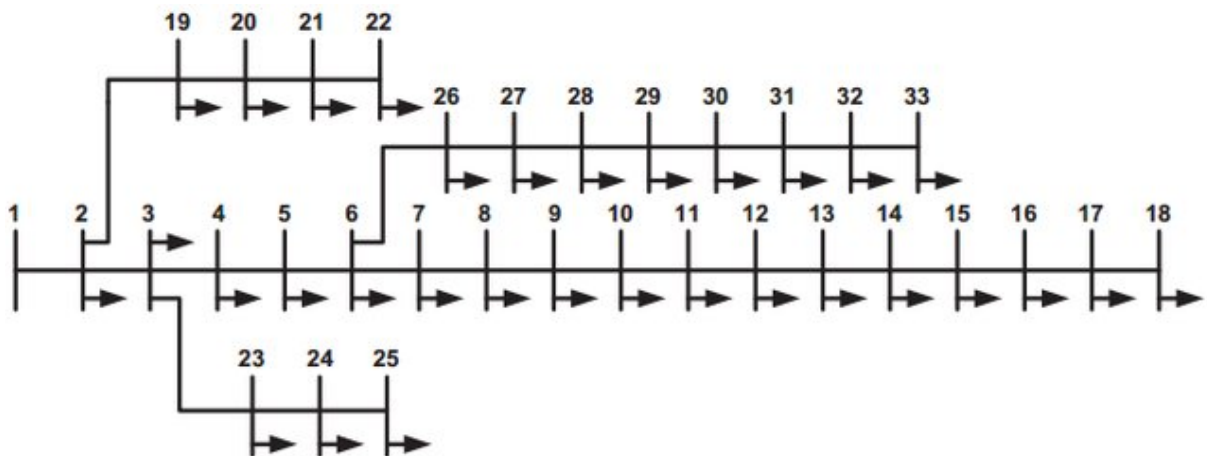


Figura 1. A topologia é puramente radial com uma barra de suprimento (barra 1, tomada como barra de referência, ou slack) e 32 barras de carga. Originalmente, a rede também inclui *tie lines*, linhas com chaves normalmente abertas, que permitem a reconfiguração de sua topologia, mas que foram desconsideradas para esta simulação. A demanda total do sistema é de 3,715 MW de potência ativa e 2,3 MVar de potência reativa, e distribui-se de forma não uniforme entre os barramentos de carga. O sistema não possui geradores distribuídos no modelo base, sendo a barra 1 a única fonte de energia. Além disso, como o foco dos autores era um algoritmo de reconfiguração, eles simplificaram a rede fornecendo apenas a impedância série

resultante, sem se preocupar com o tipo de condutor ou a distância física real. Por isso, o comprimento das linhas não foi explicitamente definido.

Entende-se que desde que o sistema de teste foi criado por Baran & Wu em 1989, as redes de distribuição passaram por significativas mudanças. Portanto melhorias têm sido propostas na literatura a fim de incluir geradores distribuídos, compensadores de potência reativa e curva de carga ao sistema, além de dados que permitem simular redes desequilibras [4]. Nesse sentido, no presente trabalho utiliza-se a rede IEEE 33 em sua configuração original apenas para validação inicial do ambiente de simulação proposto. Nas etapas subsequentes adaptações no sistema de teste, baseadas na literatura, serão realizadas para fornecer um sistema mais robusto e verossímil em relação a uma rede real.

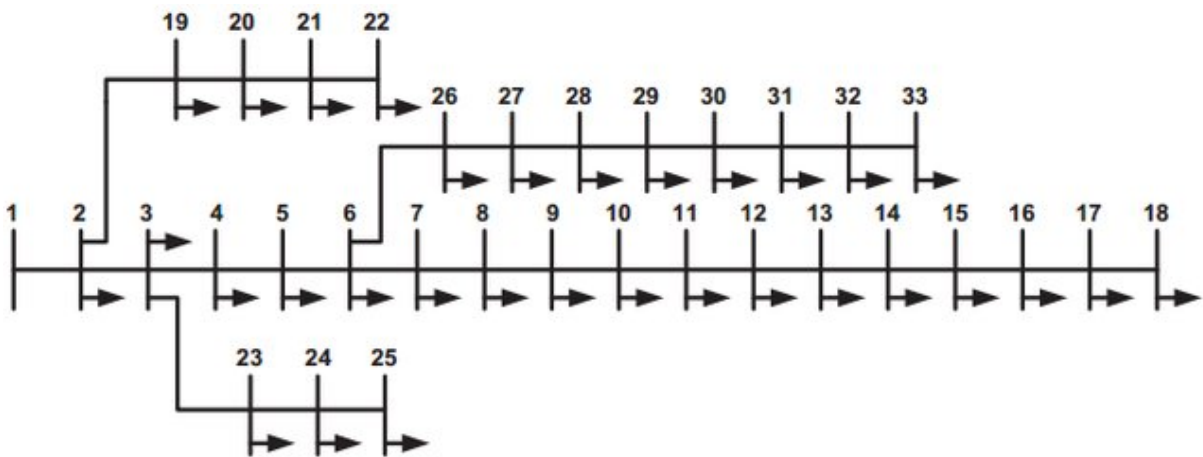


Figura 1: Sistema de testes IEEE 33 barras original

Utilizando a interface Python-OpenDss descrita foi feito uma simulação do sistema de teste IEEE 33 barras. Essa simulação foi feita com a rede operando em um cenário estático com 100% de sua carga nominal (3,715 MW) e sem simulação de série temporal (modo snapshot).

Os resultados de tensão apresentados na Tabela 1 revelam um perfil de tensão decrescente ao longo dos alimentadores, comportamento esperado em redes de distribuição radiais com carga distribuída. A barra de suprimento (B1) apresenta tensão de 0,9984 p.u., valor próximo ao nominal, enquanto ao longo do alimentador principal as tensões decrescem progressivamente até atingir valores inferiores ao limite regulatório de 0,95 p.u. recomendado pela norma brasileira PRODIST. Ao todo, um total de 20 barras opera abaixo desse limite, concentradas principalmente nos trechos mais distantes da subestação.

No alimentador principal (barras B1 a B18), a queda de tensão é acentuada especialmente a partir da barra B6, onde se inicia o ramal lateral em direção às barras B26–B33. Nesse trecho, a barra B18 apresenta a menor tensão dentre todas as barras da rede, registrando 0,9059 p.u. No ramal lateral B6–B26–B33, o afundamento de tensão também é significativo: a barra B26 já se encontra em 0,9473 p.u. e as barras B28 a B33 registram tensões entre 0,9338 e 0,9173 p.u. As barras B24 e B25 do ramal B3–B23–B25 apresentam tensões de 0,9713 e 0,9680 p.u. respectivamente, marginalmente acima do limite de 0,95 p.u.

Tabela 1: Tensões nas barras na fase A

Barra	V (pu)	Tensão de base (kV)	Barra	V (pu)	Tensão de base (kV)
B1	0,9984	7,309	B18	0,9059	7,309
B2	0,9954	7,309	B19	0,9949	7,309
B3	0,9816	7,309	B20	0,9913	7,309
B4	0,9743	7,309	B21	0,9906	7,309
B5	0,9671	7,309	B22	0,99	7,309
B6	0,9492	7,309	B23	0,978	7,309
B7	0,9458	7,309	B24	0,9713	7,309
B8	0,9327	7,309	B25	0,968	7,309
B9	0,9267	7,309	B26	0,9473	7,309
B10	0,9212	7,309	B27	0,9449	7,309
B11	0,9204	7,309	B28	0,9338	7,309
B12	0,919	7,309	B29	0,9259	7,309
B13	0,9132	7,309	B30	0,9224	7,309
B14	0,911	7,309	B31	0,9184	7,309
B15	0,9097	7,309	B32	0,9176	7,309
B16	0,9084	7,309	B33	0,9173	7,309
B17	0,9065	7,309			

A Tabela 2 apresenta as correntes e fluxos de potência ativa e reativa nas linhas do sistema. A corrente mais elevada ocorre na linha 1–2, que drena toda a demanda da rede, com 206,99 A e um fluxo de potência ativa de 1.284,39 kW. Ao se avançar no alimentador principal, as correntes diminuem progressivamente à medida que as cargas são atendidas. O ramal B6–B26 apresenta valores relativamente elevados de potência reativa, refletindo o impacto das cargas com elevada demanda reativa nesse trecho, em especial a barra B30, que possui carga de 200 kW e 600 kVAr. Essa

característica implica maior carregamento das linhas nesse ramal e contribui de forma significativa para o afundamento de tensão observado nas barras B26 a B33.

Tabela 2: Correntes e potências por fase nos elementos de rede na fase A

Linha	De	Para	Fases	I (A)	P (kW)	Q (kVAR)
source	B1	B1	3	206,99	-1284,39	-794,91
1-2	B1	B2	3	206,99	1284,39	794,91
2-3	B2	B3	3	183,74	1126,73	719,51
3-4	B3	B4	3	131,18	766,91	545,56
4-5	B4	B5	3	124,44	720,62	515,76
5-6	B5	B6	3	121,32	694,72	502,82
6-7	B6	B7	3	56,83	355,0	171,6
7-8	B7	B8	3	46,06	287,74	136,33
8-9	B8	B9	3	35,21	217,9	100,67
9-10	B9	B10	3	32,18	197,02	93,28
10-11	B10	B11	3	29,18	176,6	86,12
11-12	B11	B12	3	26,62	161,95	76,47
12-13	B12	B13	3	23,32	142,43	65,21
13-14	B13	B14	3	20,05	122,64	53,56
14-15	B14	B15	3	13,39	84,62	28,13
15-16	B15	B16	3	10,57	65,68	24,95
16-17	B16	B17	3	7,59	46,81	18,68
17-18	B17	B18	3	4,63	28,03	12,41
2-19	B2	B19	3	18,1	120,38	53,42
19-20	B19	B20	3	13,59	90,32	40,11
20-21	B20	B21	3	9,06	60,05	26,59
21-22	B21	B22	3	4,53	30,01	13,29
3-23	B3	B23	3	48,53	313,17	152,21
23-24	B23	B24	3	43,74	282,11	134,88
24-25	B24	B25	3	21,91	140,41	66,93
6-26	B6	B26	3	63,38	307,67	314,21
26-27	B26	B27	3	60,52	286,85	305,52
27-28	B27	B28	3	57,68	265,81	296,72
28-29	B28	B29	3	55,04	242,38	287,04
29-30	B29	B30	3	48,8	200,83	262,16
30-31	B30	B31	3	22,39	134,95	67,58
31-32	B31	B32	3	14,49	86,4	44,73
32-33	B32	B33	3	3,43	19,18	12,73

4. CONCLUSÕES

Nesta etapa, foi configurado com sucesso o ambiente de simulação de redes de distribuição de energia elétrica, com base na API OpenDSSDirect.py e na rede padrão IEEE 33 barras. A análise das APIs disponíveis para o OpenDSS fundamentou a escolha da OpenDSSDirect.py como biblioteca de integração, em razão de sua maturidade, documentação e adequação às demandas computacionais previstas para o projeto. O script desenvolvido demonstrou a correta obtenção dos resultados de fluxo de potência (tensões nodais, correntes nas linhas e fluxos de potência ativa e reativa), validando o ambiente de simulação e estabelecendo a infraestrutura necessária para o desenvolvimento das etapas de controle previstas nas fases seguintes do plano de trabalho.

BIBLIOGRAFIA

- [1] LACERDA, Rafaela; MORAIS, Ana Júlia de. projeto_de_formatura: configuracao_de_ambiente/my_power_flow.py. [S. l.]: GitHub, 2026. 1 repositório. Disponível em: https://github.com/rafaxlacerda/projeto_de_formatura/blob/main/configuracao_de_ambiente/my_power_flow.py. Acesso em: 23 abr. 2026.
- [2] LACERDA, Rafaela; MORAIS, Ana Júlia de. projeto_de_formatura: configuracao_de_ambiente/IEEE33.dss. [S. l.]: GitHub, 2026. 1 repositório. Disponível em: https://github.com/rafaxlacerda/projeto_de_formatura/blob/main/configuracao_de_ambiente/IEEE33.dss. Acesso em: 23 abr. 2026.
- [3] M. E. Baran and F. F. Wu, "Network reconfiguration in distribution systems for loss reduction and load balancing," IEEE Transactions on Power Delivery, vol. 4, no. 2, pp. 1401-1407, Apr. 1989, doi: 10.1109/61.25627.
- [4] S. H. Dolatabadi, M. Ghorbanian, P. Siano, and N. D. Hatziargyriou, "An Enhanced IEEE 33 Bus Benchmark Test System for Distribution System Studies," IEEE Transactions on Power Systems, vol. 36, no. 3, pp. 2565-2572, May 2021, doi: 10.1109/TPWRS.2020.3038030.